

JavaScript Export & Import Guide

Easy English — Beginner Friendly

1. What is a Module?

A **module** is simply a JavaScript file (.js). Think of it like a box — you put some code inside it, and you can share that code with other files. One file can **export** (share) things, and another file can **import** (use) them.

2. Export — Sharing Your Code

When you write something in a file and want other files to use it, you **export** it. There are two ways to export:

A) Named Export

You give a name to what you are sharing.

```
// file.js
export const name = 'Alice';
export function greet() {
  return 'Hello!';
}
```

✓ TIP: You can export many things from one file using named exports.

B) Default Export

Each file can have **one** default export — the main thing it shares.

```
// file.js
export default function greet() {
  return 'Hello!';
}
```

■ NOTE: A file can only have ONE default export, but many named exports.

3. Import — Using Shared Code

When you want to use code from another file, you **import** it.

A) Import Named Exports

```
// main.js
import { name, greet } from './file.js';

console.log(name);    // Alice
console.log(greet()); // Hello!
```

B) Import Default Export

```
// main.js
import greet from './file.js';

console.log(greet()); // Hello!
```

C) Import Everything

```
// main.js
import * as myFile from './file.js';

console.log(myFile.name);    // Alice
console.log(myFile.greet()); // Hello!
```

4. Dynamic Import — The Star of This Guide

Normal imports happen at the **top of the file**, before anything runs. But sometimes you want to load a file **only when needed** — maybe when a user clicks a button. That is called a **dynamic import**.

The Syntax:

```
const mod = await import('./file.js');
```

Part	Meaning
const mod	Store the imported file in a variable called mod
await	Wait for the file to finish loading before moving on
import(...)	Load the file dynamically (only when this line runs)
'./file.js'	Path to the file you want to load

Full Example:

```
// file.js
export const name = 'Alice';
export default function greet() {
  return 'Hello!';
}

// main.js
```

```

async function loadModule() {
  const mod = await import('./file.js');

  // Access named export
  console.log(mod.name);           // Alice

  // Access default export
  console.log(mod.default());      // Hello!
}

loadModule();

```

✓ TIP: Dynamic import returns a Promise. Use await (inside async function) or .then() to get the module.

5. Static vs Dynamic Import — Quick Comparison

Feature	Static Import	Dynamic Import
Syntax	import x from './f.js'	await import('./f.js')
When it loads	At start, always	Only when that line runs
Can use conditions?	No	Yes (inside if, onClick, etc.)
Returns	The value directly	A Promise
Use case	Regular imports	Lazy loading, code splitting

6. Real-World Use Case

Imagine a website where a heavy chart library is only needed when the user clicks 'Show Chart'. Instead of loading it for everyone, you load it dynamically:

```

button.addEventListener('click', async () => {
  const chartLib = await import('./chartLibrary.js');
  chartLib.drawChart(data);
});

```

✓ TIP: This makes your website faster — it only loads what is needed!

Quick Recap

Keyword	What it does
export const x	Share a variable named x
export default	Share the main thing from a file
import { x }	Get named export x from a file
import x	Get the default export

<code>import *</code>	Get everything from a file
<code>await import()</code>	Load a file dynamically when needed

You are now ready to use export and import in JavaScript! Start with named exports, then try dynamic import() for bigger projects.